

UCS503 – Software Engineering Lab

**SHARP: A Smart Hostel
Administration and Resource Platform**
Use Case and UML Design for Digital Hostel Workflows

Submitted by:

1024160097 Vaibhav Kansal

1024160138 Ali Ekram

1024160122 Riya Gupta

1024160118 Ayush Kamboj

Group: 3P14, BE Third Year, CSE

Submitted to:

Ms. Anushka

PROJECT REPORT

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	5
2.2	System Users and Use Cases	6
2.3	Use Case Diagram	6
2.4	Class Diagram	7
2.5	Sequence Diagram: Complaint Lifecycle	7
2.6	Activity Diagram: Complaint Lifecycle	10
2.7	Implementation Modules	10
2.8	Risk Management	13
2.9	Prototype	13
3	Results and Evaluation	17
3.1	Testing Methodology	17
3.2	Evaluation Metrics	17
3.3	Pilot Validation Plan	17
3.4	Analysis	18
4	Conclusion	20
4.1	Summary of Work	20
4.2	Key Findings	20
4.3	Future Work and Recommendations	20
4.4	Final Remarks	21
	Bibliography	22

Introduction

1.1 Background and Context

Hostel administration at Thapar Institute of Engineering and Technology currently depends on a mix of handwritten applications, physical registers, and informally tracked complaints spread across five recurring processes: leave and visitor passes, mess feedback, room allotment, maintenance complaints, and laundry issue reporting. None of these processes currently produce an auditable digital record, which makes it difficult to measure resolution time, detect recurring problems, or settle disputes between students and hostel staff.

SHARP (Smart Hostel Administration and Resource Platform) is proposed as a single web-based system that converts these five processes into structured, traceable digital workflows. The immediate engineering objective is not novel algorithm research but *reliable, verifiable service delivery*: every request should carry a unique identifier, a defined owner, and a timestamped history from creation to closure. The system is intended to scale from a single pilot hostel wing to the full campus without a redesign, by keeping each of the five service modules independent behind a common authentication and role-based access layer.

1.1.1 Problem Statement

Five specific pain points motivate the system:

- **Leave and passes:** paper applications and manual parent contact create delay and are easy to lose or misrepresent.
- **Mess feedback:** register-based feedback is visible to mess staff, which discourages honest reporting.
- **Allotment:** when members of the same room cluster pay at different times, a partially reserved room can be taken by someone else, breaking up the cluster.
- **Maintenance:** complaints have no owner or deadline once filed, so resolution time and preferred repair slots go untracked.
- **Laundry:** damaged or missing items are reported informally, so recurring problems by day, batch, or item type go unnoticed.

Because hostel staff bandwidth is limited relative to the student population, even small structural improvements – a unique complaint ID, an allotment expiry rule, a

Service Level Agreement (SLA) timer – are expected to measurably reduce unresolved cases and disputes.

1.2 Project Objectives

1. **Role-based access first:** deliver authentication and role-based access control for all six user roles (Student, Parent, Warden, Caretaker, Mess Staff, Laundry Staff) before any service module, so every subsequent workflow is access-controlled from the start.
2. **Leave workflow as first vertical slice:** implement the leave and visitor pass workflow end to end (Student → Parent verification → Warden approval → digital pass) as the first complete, demonstrable path through the system.
3. **Complaint lifecycle as second vertical slice:** implement complaint filing, caretaker assignment, SLA-timed escalation, and student-verified closure, since this workflow is the primary source of the project's evaluation metrics.
4. **Model before building:** fix the user/functionality boundary with a use case diagram, and fix the data model and two representative workflows with UML class, sequence, and activity diagrams, before implementation of each module begins.
5. **Independent, deployable modules:** design every module so the mess, allotment, and laundry modules can be added in later iterations without redesigning the authentication, complaint, or leave modules already in place.

Objectives 1–3 are Specific, Measurable (each has a clear done/not-done boundary), Achievable within a lab-scale project, Relevant to the problem statement above, and Time-bound by the phased delivery plan in Section 2.7.

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

SHARP follows a five-tier layered architecture rather than a monolithic design, so that any one concern can change without forcing a redesign of the others:

- **Interface layer:** role-specific portals for students/parents, wardens/admins, and caretaker/mess/laundry staff.
- **Application layer:** REST API endpoints, authentication (hashed credentials, tokens), and role-based access control.
- **Application services layer:** the five service modules (leave & pass, mess feedback, allotment, complaints & SLA, laundry issues) plus notifications and analytics.
- **Data layer:** users, requests, complaints and reports, status history, audit records, backup and recovery.
- **External services layer (optional):** email, SMS, or push notification providers.

Every sensitive endpoint checks both identity and permission, so, for example, a student cannot reach warden dashboards, parent contact records, or the identity behind an anonymous mess feedback entry.

2.1.2 Technical Stack

The layers above are specified by responsibility rather than by product, so the exact implementation stack is finalised at build time. The current working assumption is:

- **Frontend:** React with Tailwind CSS for role-specific reactive pages.
- **Backend:** Node.js with Express.
- **Database:** MongoDB.
- **Authentication:** token-based authentication with hashed credentials.
- **Testing:** a unit/integration test framework exercised on every change.

- **CI/CD:** a CI runner with staging deployment.

No workflow rule, role definition, or evaluation metric in this report depends on this specific stack, so deferring the final choice costs nothing structurally.

2.2 System Users and Use Cases

Six users interact with SHARP. Table 2.1 identifies each user and the use cases it participates in; these are the same use cases drawn in Figure 2.1.

User	Role	Primary Use Cases
Student	Primary end user; a hostel resident	Login/Authenticate, Apply for Leave/Pass, Submit Mess Feedback, Request Room Allotment, Make Payment, File Complaint, Verify Complaint Resolution, Report Laundry Issue, Track Request Status
Parent	Secondary human user; verifies a leave request for their child	Verify Leave Request
Warden	Hostel administrator for a block	Approve Leave Request, Assign Caretaker, Handle SLA Escalation, View Analytics Dashboard
Caretaker	Maintenance staff	Update Complaint Status
Mess Staff	Mess operations staff	View Aggregated Mess Feedback
Laundry Staff	Laundry operations staff	Update Laundry Issue Status

Table 2.1: Users and their primary use cases

2.3 Use Case Diagram

Figure 2.1 follows the OMG UML 2.5.1 notation [Obj17]: users are drawn outside the system boundary, use cases are ellipses inside it, and «include»/«extend» dependencies capture mandatory and optional sub-behaviour. Three relationships are worth calling out explicitly, since they encode workflow rules from Section 2.2 directly into the model:

- *Apply for Leave/Pass* includes *Verify Leave Request*, which itself includes *Approve Leave Request* – the same three-step approval chain (Student → Parent → Warden) described in Section 2.7.

- *File Complaint* includes *Assign Caretaker* (assignment is mandatory on every complaint) but is only *extended* by *Handle SLA Escalation*, since escalation is conditional on the SLA timer actually being breached.
- *Request Room Allotment* includes *Make Payment*, since a cluster reservation is not confirmed until payment completes.

Login/Authenticate is included by every primary student transaction; only one representative edge is drawn in the diagram to keep it legible.

2.4 Class Diagram

Figure 2.2 is the class diagram produced for SHARP’s domain model. **User** is an abstract base class carrying shared authentication attributes (`userId`, `name`, `email`, `password`) and behaviour (`login()`, `logout()`); the six user roles are modelled as subclasses connected to **User** by generalization (hollow-triangle arrows). Each role carries only the attributes and methods specific to it – for example **Warden** adds `approveLeave()`, `manageAllotment()`, `monitorSLA()`, and `viewReports()`, while **Caretaker** adds `manageComplaint()` and `updateRepairStatus()`.

Below the user layer, the diagram groups the domain classes into the five service modules from Section 2.7: *Leave Management* (**LeaveRequest**, which generates at most one **DigitalPass** once both `parentVerified` and `wardenApproved` are true – a 1.0..1 association), *Maintenance Management* (**Complaint**, with `slaDeadline` and `create()/assign()/updateStatus()/verifyResolution()/close()/reopen()`), *Room Allotment* (**RoomReservation**, with `reserve()/confirm()/expire()` implementing the cluster-expiry rule from the proposal), *Mess Management* (**MessFeedback**), and *Laundry Management* (**LaundryIssue**). **Student** is associated 1.0..* with all five module classes, since a student is the one user who creates every kind of request; **Parent** and **Warden** are associated with *Leave Management* (verifying and approving a **LeaveRequest** respectively); **Warden** and **Caretaker** are both associated with *Maintenance Management*, matching `monitorSLA()` and `manageComplaint()`; and **MessStaff**/**LaundryStaff** are associated with their respective modules only.

2.5 Sequence Diagram: Complaint Lifecycle

Figure 2.3 traces the complaint lifecycle as message exchanges among five lifelines: the **Student**, **Caretaker** and **Warden** users, the **SHARP System** that mediates every request, and the **Complaint Database** it persists to. The system, not the users, owns every write to the database: a complaint record is created and given an ID as soon as it is filed, the caretaker’s repair status is saved as it changes, and the closing or reopening of a complaint is itself a database write triggered by the system

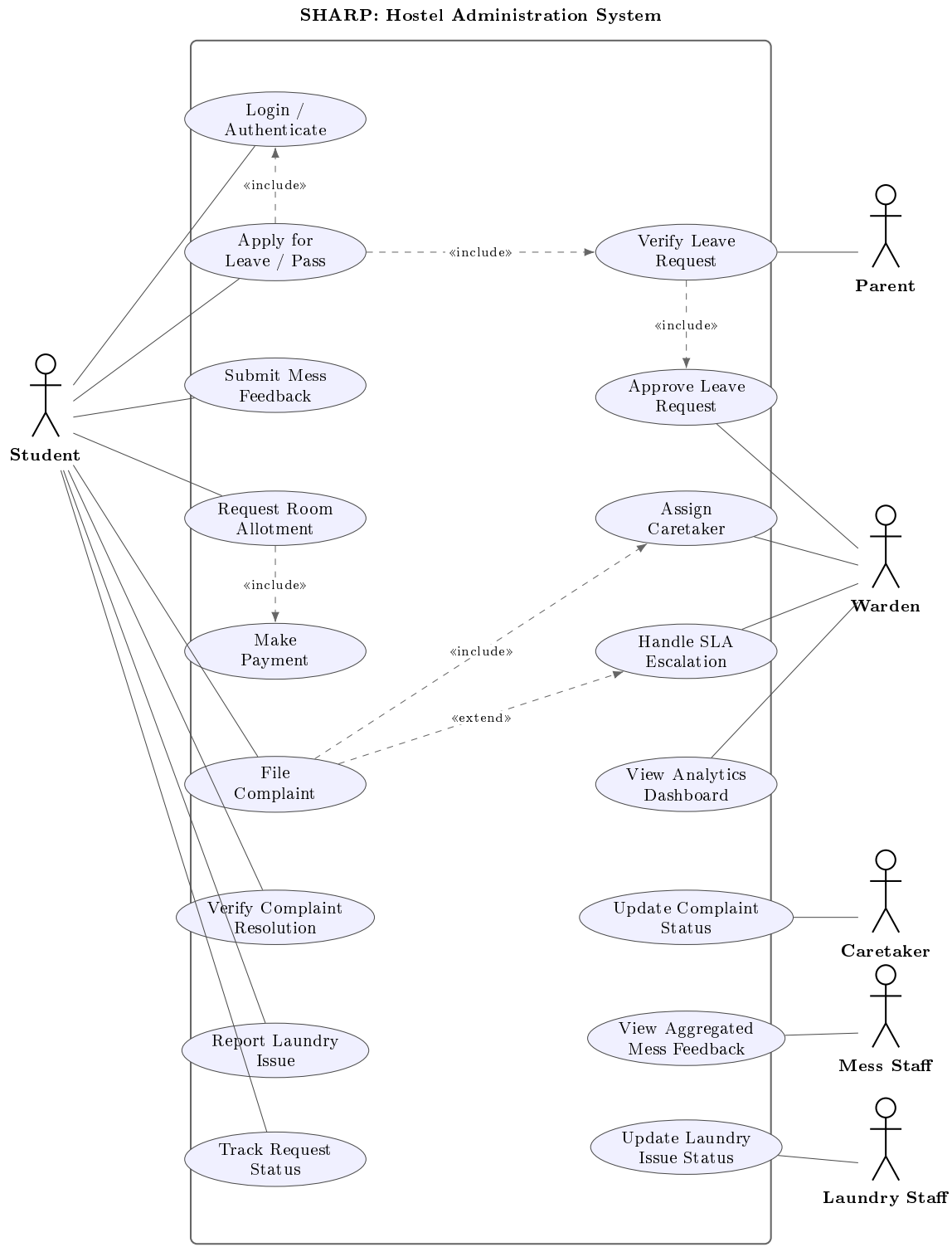


Figure 2.1: Use Case Diagram: SHARP Hostel Administration System

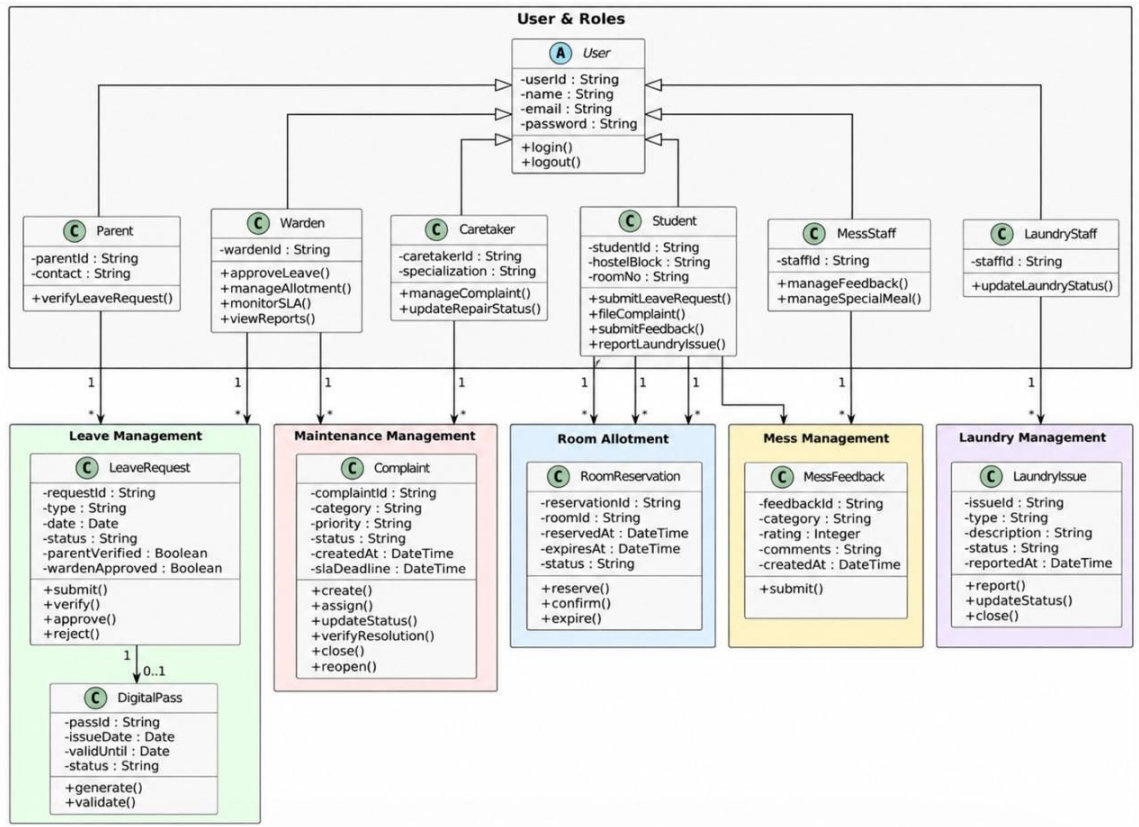


Figure 2.2: Class Diagram: SHARP core domain model

after the student’s verification message. Two `alt` frames capture the workflow’s two decision points – whether the student verifies the resolution (closing or reopening the same complaint ID) – while a second, independent `alt` [SLA breached] frame sends an escalation notification to the Warden without being on the critical path of the student-facing flow at all. This matches the closure rule in Section 2.7: a failed verification reopens the existing complaint rather than creating a new one.

2.6 Activity Diagram: Complaint Lifecycle

Where Figure 2.3 shows the complaint lifecycle as messages between users and the system, Figure 2.4 shows the same lifecycle as a single control flow, which is the clearer view of the two decision points once who-sends-what stops being the question. After a complaint is filed, its ID and priority are generated and a caretaker is assigned automatically. The first decision, ‘Repair slot agreed?’, has both branches – accepting the proposed slot, or the caretaker proposing an alternative – merge back into a single ‘Repair completed’ step, since the diagram treats slot negotiation as a detail that does not change what happens next. The second decision, ‘Resolution verified?’, closes or reopens the complaint. Only after that merge does the diagram check ‘SLA breached?’: this ordering places the SLA check after resolution rather than racing it against the main flow, which is a simpler model than the sequence diagram’s independent `alt` frame, at the cost of not showing that the SLA timer can in practice breach and notify the Warden *before* the complaint is resolved. The two diagrams are deliberately kept as complementary views rather than reconciled into one, since each makes a different property of the workflow easier to see.

2.7 Implementation Modules

The four diagrams above cover the full system, but delivery is phased. The initial deliverable is:

- Authentication and role-based access control for all six roles.
- The leave and visitor pass workflow (Student → Parent → Warden → digital pass).
- The complaint lifecycle (filing, assignment, SLA timer, verified closure) – modelled in Figure 2.3.
- Basic logging and timestamps to support the resolution-time metric in Section 3.2.
- CI: build, backend unit tests, and linting on every change.

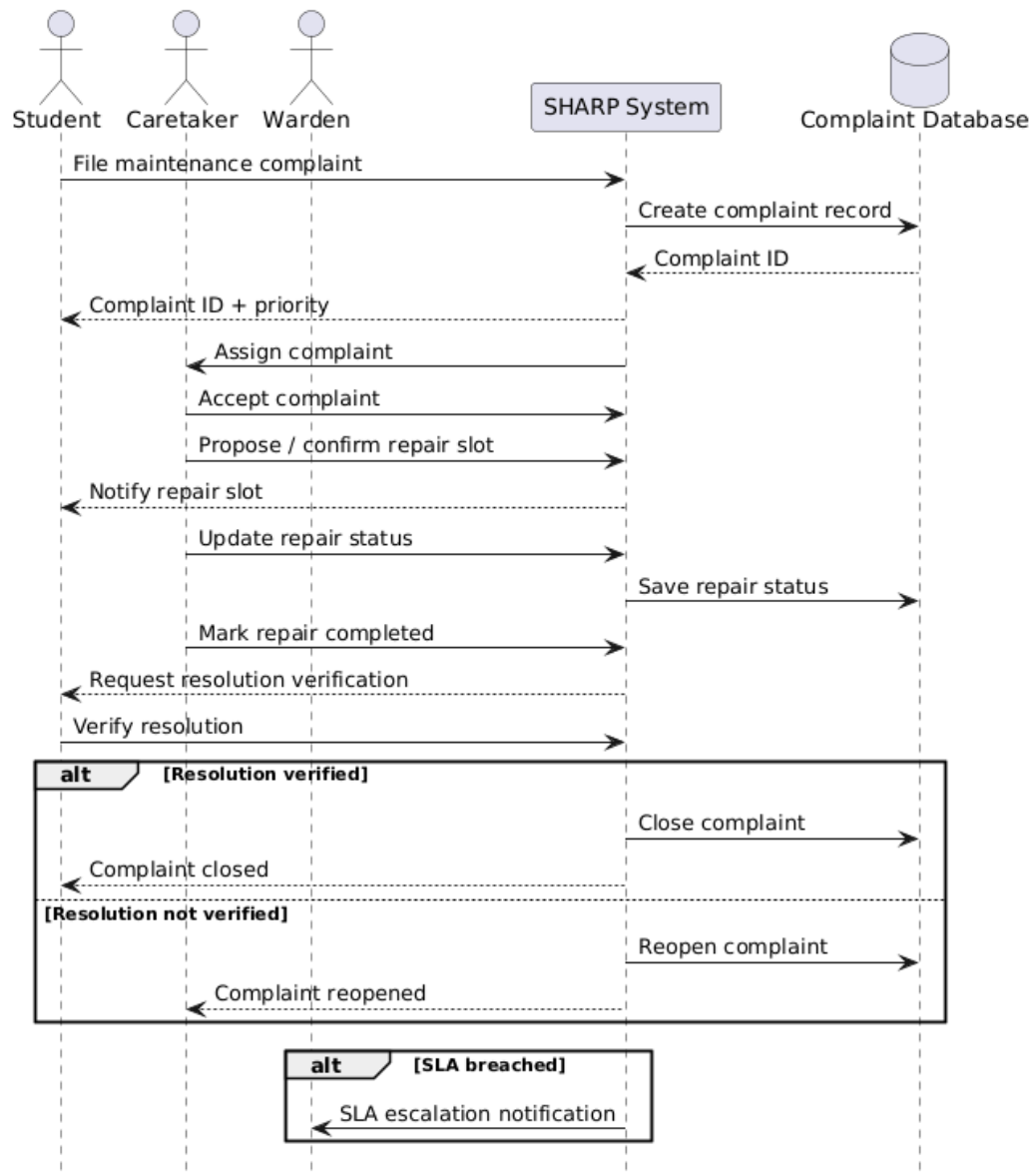


Figure 2.3: Sequence Diagram: Complaint Lifecycle

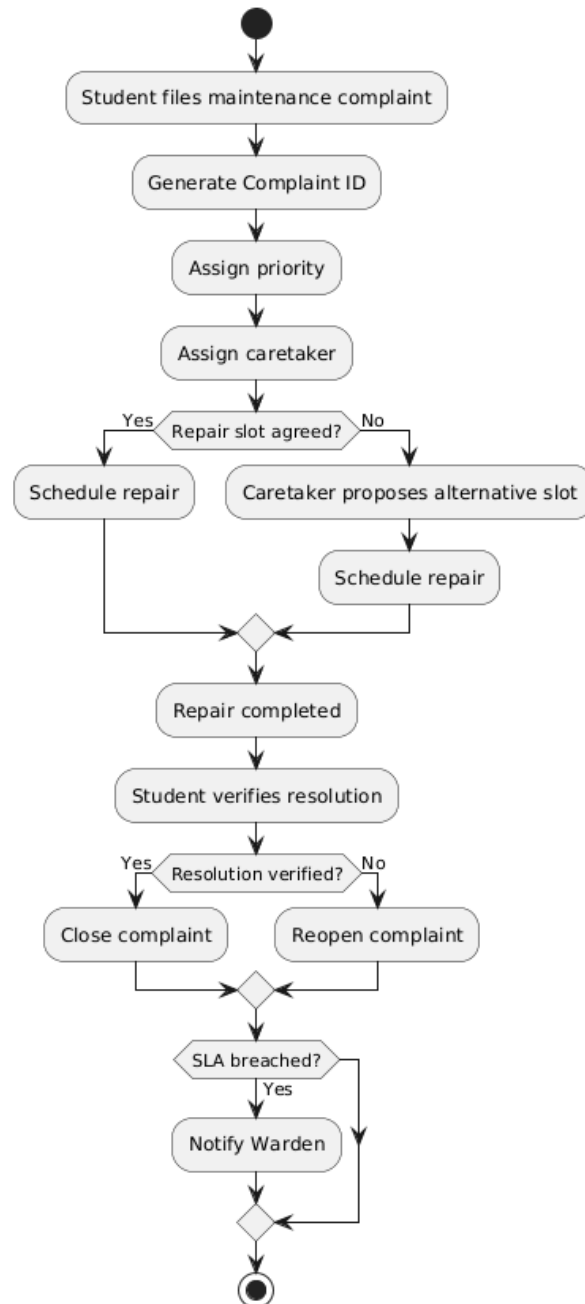


Figure 2.4: Activity Diagram: Complaint Lifecycle

- CD to a staging environment from a single pipeline job.

Subsequent deliverables, built on the same class model (Figure 2.2) and layered architecture, are the mess feedback module (anonymous to staff, with internal abuse-deterrence records), the allotment module (temporary reservation with expiry), laundry issue tracking with recurring-issue aggregation, an in-app notification layer, and a warden/admin analytics dashboard.

2.8 Risk Management

Risks were identified against the design in this chapter rather than left for the implementation phase:

- **Unauthorised access** is mitigated by the role-based access control enforced at the application layer for every request.
- **Forged parent approval** is mitigated by secure, single-use verification tokens combined with a subsequent warden check, rather than relying on parent verification alone.
- **Abuse of anonymous mess feedback** is mitigated by rate limits and an internal audit trail that is not exposed to mess staff.
- **Premature loss of a room reservation** is mitigated by the `Allotment` class's expiry rule and transactional payment validation.
- **Disputed laundry loss** is mitigated by timestamped `LaundryIssue` reports tied to a specific submission.

2.9 Prototype

A working prototype of SHARP has been built to check the design above against a real interface, currently covering the debug/demo authentication flow and both the caretaker- and student-facing sides of the room allocation console for Phase 2 (Cluster Formation) of the allotment module. Figure 2.5 shows the landing page: a demo role switcher bypasses manual login for testing, and the page reports campus-wide capacity figures (11 hostels, 8 floors per hostel, 50 rooms per floor, 8,800 total bed capacity) that the schema in Figure 2.2 is sized against.

On the caretaker side, Figure 2.6 shows the caretaker's dashboard scoped to a single assigned facility (Hostel M, Anantam Hall), consistent with the role-based access rule in Section 2.1.1: a caretaker sees only the hostel block they are assigned to, not the full campus. Figure 2.7 shows the caretaker's room allocation console: each

candidate cluster lists its members by roll number, average CGPA, eligible hostels in preference order, and a status of **FORMED**, **ALLOCATED**, or **CONFIRMED** – a direct implementation of the cluster-preservation rule from the original problem statement, where a cluster is only finalised once every member’s preferences and payment status agree, rather than allocating rooms to individual students as they pay.

On the student side, Figure 2.8 shows the same role-scoped pattern from the other direction: a student sees only their own room, resident ID, and allocation status, with room allocation and room/hostel info as the only two actions exposed for this phase. Figure 2.9 shows what a cluster leader sees when they open that action: the same cluster (members, average CGPA, eligible hostels) the caretaker sees in Figure 2.7, but from the student’s side, with controls to submit room-pair preferences or change the cluster leader rather than to allocate or resolve conflicts. Together the two pairs of screens show the same **RoomReservation** data (Figure 2.2) correctly restricted to what each role is allowed to do with it.

This prototype currently demonstrates the **RoomReservation** portion of the class diagram (Figure 2.2); the leave and complaint vertical slices described in Section 2.7 are the next modules to be connected to a similar interface.

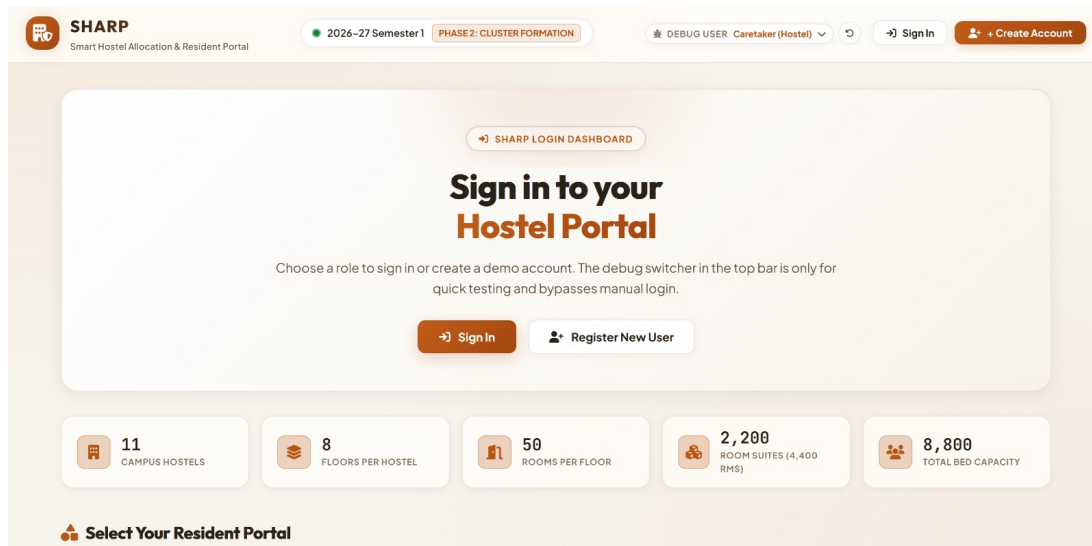


Figure 2.5: Prototype: landing page and role selection

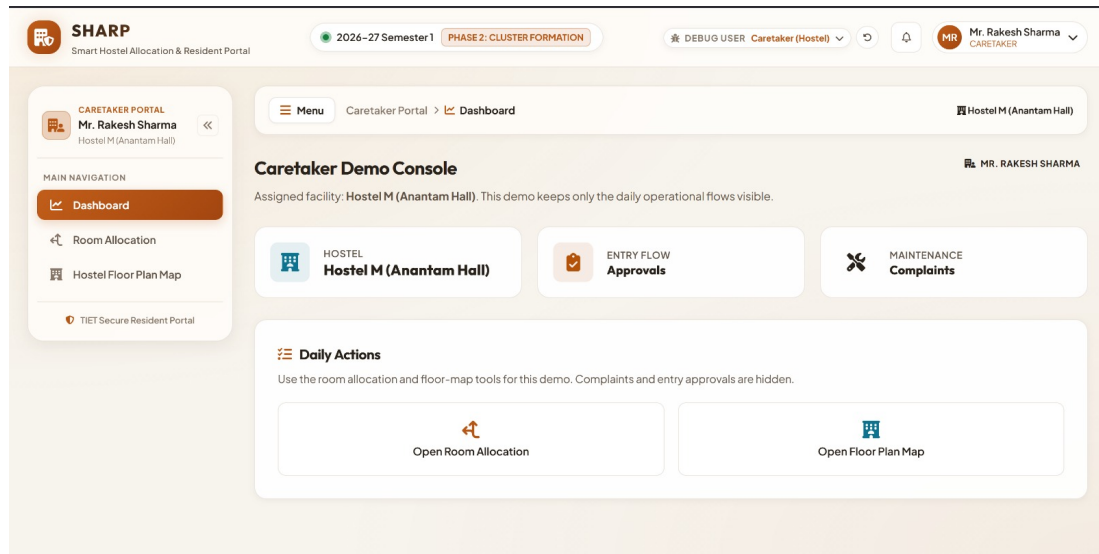


Figure 2.6: Prototype: caretaker dashboard, scoped to one hostel block

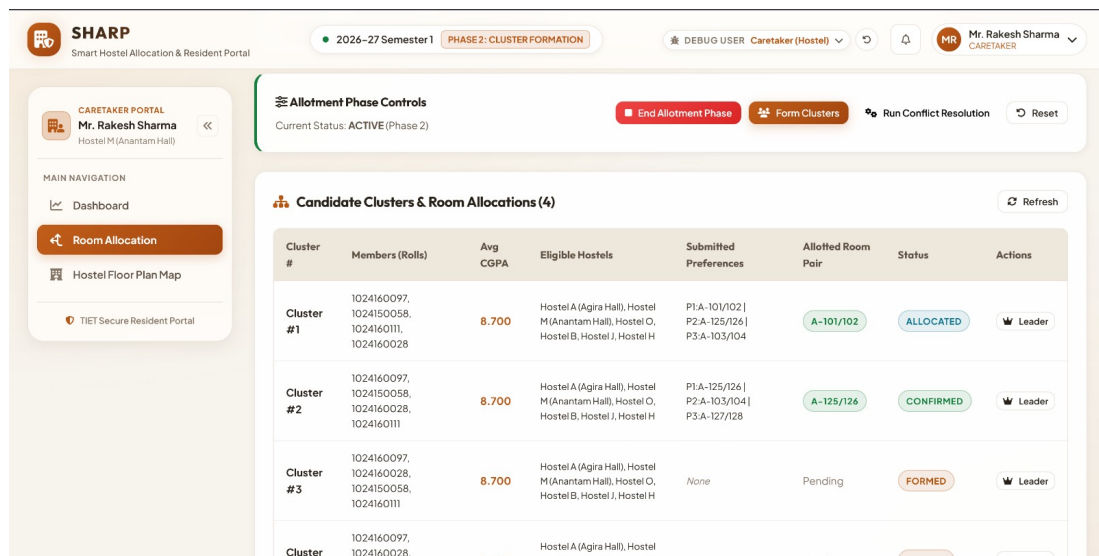


Figure 2.7: Prototype: caretaker's room allocation and cluster formation console

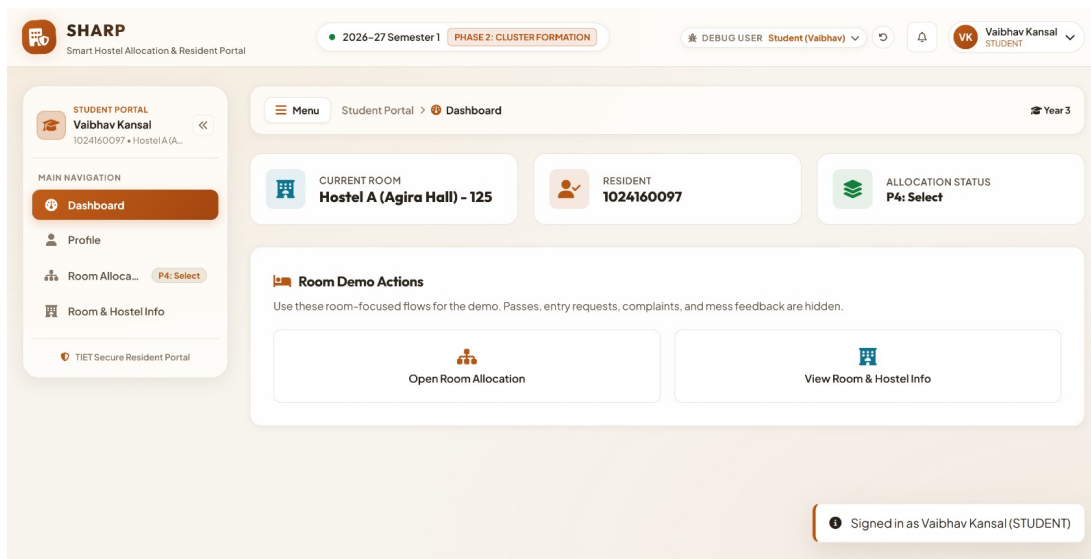


Figure 2.8: Prototype: student dashboard, scoped to one resident

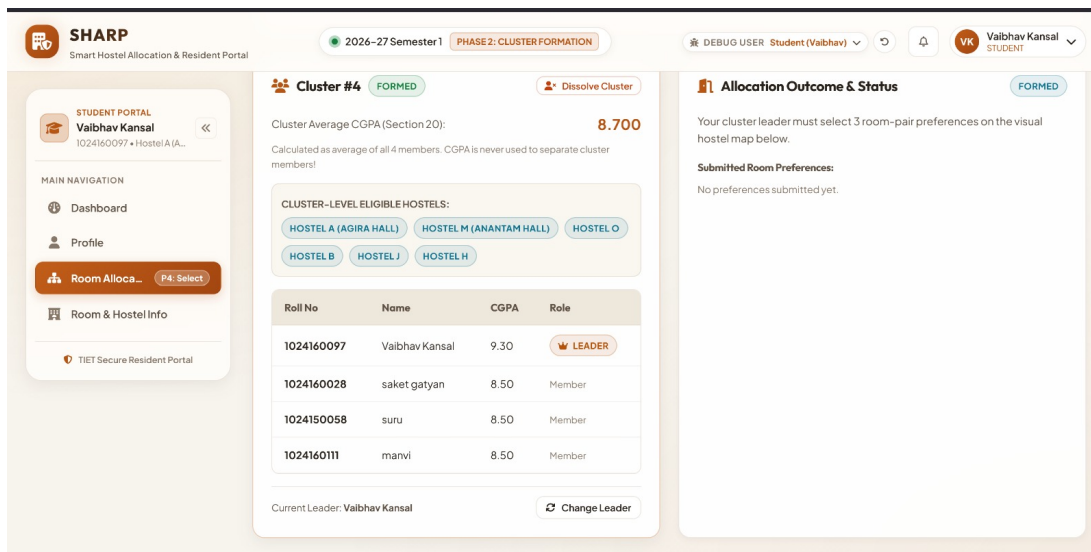


Figure 2.9: Prototype: student-facing cluster and room preference view

Results and Evaluation

This report documents SHARP partway between design and implementation: the use case and UML diagrams in Chapter 2 are complete, and a first prototype (Section 2.9) demonstrates the room allocation console end to end. The leave and complaint vertical slices, which carry the metrics below, are still in progress. This chapter therefore records the *evaluation plan* that will be executed once those slices are deployed to staging, not measured results.

3.1 Testing Methodology

- **Unit testing:** each application-service method identified in Figure 2.2 (for example `Complaint.updateStatus()`, `RoomReservation.expire()`) will be tested in isolation.
- **Integration testing:** the full message sequence in Figure 2.3 will be exercised against a staging database to confirm status transitions match the alt branches modelled, cross-checked against the equivalent decision points in Figure 2.4.
- **Performance testing:** concurrent request handling will be measured under simulated multi-hostel load once the schema is confirmed to admit further hostels without redesign.
- **User testing:** the pilot plan in Section 3.3 exercises the system with real students, a warden, a caretaker, and mess/laundry staff.

3.2 Evaluation Metrics

All measurements are attributed from database timestamps (filing, assignment, completion, verification) rather than self-reported time, so that the metrics in Table 3.1 remain auditable.

3.3 Pilot Validation Plan

A pilot hostel wing will be recruited covering all six roles: students, a warden, a caretaker, and mess and laundry staff. Baseline estimates will be logged from existing

Metric	Definition
<i>Primary</i>	
Complaint resolution time	Time between complaint filing and student-verified closure, from database timestamps.
SLA compliance	Share of complaints closed within the defined SLA window.
<i>Secondary</i>	
Paper reduction	Reduction in paper-based leave applications after deployment.
Leave approval time	Average time from student request to digital pass issuance.
Reopen count	Number of complaints reopened after a failed verification.
Laundry issues per month	Volume reported, and recurring-issue detection rate.
Reservation conflicts avoided	Clusters preserved through the expiry-based allotment mechanism.

Table 3.1: Planned evaluation metrics

manual records (paper leave applications, the current complaint register) before deployment, so that Table 3.1 can be compared before/after within the same iteration window rather than against an assumed baseline.

3.4 Analysis

Measured against the objectives in Section 1.2:

- **Objectives 1 and 4** (role-based access first; model before building) are met by this report: Table 2.1 and Figure 2.1 fix the user/permission boundary, and Figure 2.2 fixes the schema each role reads and writes. The prototype in Section 2.9 already enforces one instance of this rule in a real interface: a caretaker’s dashboard is scoped to a single hostel block.
- **Objective 3** (complaint lifecycle as a vertical slice) is modelled twice, deliberately: Figure 2.3 shows it as messages between the users, the system, and the database, and Figure 2.4 shows the same workflow as a single control flow. Neither has been implemented yet, so neither is measurable against Table 3.1 for now.
- **Objective 2** (leave workflow as a vertical slice) has no dedicated sequence or activity diagram in this report – it remains fixed only structurally, through the

«include» chain in Figure 2.1 and the `LeaveRequest`/`DigitalPass` classes in Figure 2.2. This is a gap to close before that module is implemented, not a design decision.

- **Objective 5** (independent modules) is supported by both the layered architecture in Section 2.1.1 and, concretely, by the prototype: the room allocation console in Figure 2.7 works against the `RoomReservation` class alone, without any leave or complaint code having been written yet.
- The main open design risk is the disagreement between Figures 2.3 and 2.4 over when the SLA check happens: the sequence diagram runs it independently of the main flow, while the activity diagram checks it only after resolution. The implementation must pick one of these two semantics – an independent timer is closer to the SLA guarantee in the original proposal, so Figure 2.3’s ordering should be treated as authoritative.

Conclusion

4.1 Summary of Work

This report has translated the SHARP project proposal into a formal design: six users and seventeen use cases were identified (Table 2.1, Figure 2.1); a class model was built around an abstract `User` base class and five module-grouped domain classes (Figure 2.2); and the complaint lifecycle was modelled twice, as a UML sequence diagram (Figure 2.3) and as an activity diagram (Figure 2.4), to check the same workflow from two angles. A first prototype (Section 2.9) already implements the room allocation console end to end against this design. Together these satisfy the lab’s requirement for a use case diagram plus supporting UML diagrams appropriate to the system, while leaving a clearly identified gap – a dedicated diagram for the leave workflow – for the next iteration.

4.2 Key Findings

- **Finding 1:** the include/extend structure of the use case diagram maps directly onto the multi-step approval chains already described in the proposal (Apply → Verify → Approve; File Complaint → Assign, optionally → Escalate), suggesting the proposal’s workflow language was already close to UML use-case semantics.
- **Finding 2:** every user-specific method on the class diagram (Figure 2.2) traces back to exactly one row of Table 2.1, which is a useful design-time check that no user has been given responsibilities outside its stated role.
- **Finding 3:** the sequence and activity diagrams do not agree on when the SLA check happens, as noted in Chapter 3’s Analysis section – modelling the same workflow twice surfaced a real ambiguity in the original proposal’s prose that a single diagram would have hidden.

4.3 Future Work and Recommendations

1. Model the leave/visitor-pass workflow with its own sequence or activity diagram, closing the gap noted in Chapter 3’s Analysis section, before that module is implemented.

2. Implement the modules listed in Section 2.7 in the stated order, validating the complaint lifecycle against Figure 2.3 and Figure 2.4 and resolving the SLA-timing disagreement between them in favour of the independent-timer reading.
3. Extend the class diagram with a **Notification** class once the notification layer is scoped, since every domain class in Figure 2.2 currently triggers a notification implicitly.
4. Re-run the pilot validation plan (Section 3.3) after the first vertical slice is deployed, and replace Table 3.1's definitions with measured values.
5. Connect the leave and complaint modules to the same prototype interface as the room allocation console in Figure 2.7, rather than treating the prototype as allotment-only.
6. Evaluate deployment on managed hosting once the schema is confirmed against a second, structurally different hostel block, to validate the "admits further hostels without redesign" scalability claim from the original proposal.

4.4 Final Remarks

Modelling the complaint lifecycle twice – once as messages, once as a control flow – did more than satisfy the lab's requirement for multiple UML diagrams: it surfaced a genuine disagreement, over when the SLA check happens, that neither diagram alone would have shown. Building the room allocation console before the leave and complaint modules also shows the value of the layered architecture in Section 2.1.1 directly: a working prototype exists for one service module without the others existing yet. That is the main value of this exercise – it turns the proposal's workflow language into diagrams and a prototype that can be checked, questioned, and built on directly.

Bibliography

- [Obj17] Object Management Group. *OMG Unified Modeling Language (UML), Version 2.5.1*. Tech. rep. formal/2017-12-05. Object Management Group, 2017 (cit. on p. 6).